Day 3 Student Notes: Prompt Engineering for Infrastructure Work

Claude for DevOps Mastery. Keep these notes and the prompt template.

> **Goal of Day 3:** turn a vague request into a clear prompt, and check that the answer is safe. You learn it by comparing bad and good prompts on the same task.

Start simple: ordering food is a prompt

```
Bad: Make me some food.

Good:
Role: You are a good home cook.
Context: We are 4 people. One person does not eat onion.
Task: Make a chicken biryani.
Constraints: Medium spicy. No onion.
Format: Give me the steps in 5 short points.
```

The bad order makes the cook guess. The good order gives role, context, task, constraints and format. Claude works the same way.

The key idea: in the free chat, Claude only writes text

Claude on the free web chat can only write words and code. It cannot create a folder, run a command or change anything on your computer. You copy the code, read it, and run it yourself on a practice computer.

In a chat, Claude only knows the words you type. It cannot see your operating system, your folder names, your tool versions, or whether it is a practice machine or production. If you do not tell it, it **guesses**. In infrastructure, a wrong guess can break things. A good prompt supplies the missing facts.

RCTCF: the 5 parts of a good prompt

Remember **R-C-T-C-F**: Role, Context, Task, Constraints, Format.

Part	What you tell Claude	If you leave it out
Role	Who to act as	Answer may be too advanced or too basic
Context	Your system and situation	Claude guesses your OS, folders, versions
Task	What you want done	Claude does something else
Constraints	What it must NOT do	It may delete, overwrite or add extras
Format	How the answer looks	Long, messy answer, hard to check

Prompt template (copy and fill in)

```
Role: You are a ... (who Claude should be)
Context: About me and my situation (Claude cannot see it, so tell it)
Task: What I want
Constraints: What Claude must NOT do, or must keep
Format: How the answer should look
```

I am a beginner and not sure of my operating system. Give the answer for Linux and tell me how to check which system I have.

Why vague prompts are dangerous

```
Bad: Clean up my computer.

Good: Show me the 10 biggest files in my current folder. Only list them. Do not
delete or change anything. Explain the command in simple words.
```

The bad prompt lets Claude decide which files to delete and where. The good prompt only lists files, so you read the list and decide. Guard-rails: say "do not delete or change anything", ask for a list or preview first, and use a practice machine.

What are the three tasks?

Word	In plain English
Script	A list of computer commands saved in a file, so you can run them again and again
YAML file	A settings file. It tells a tool what to run and how. Kubernetes uses YAML
Error message	The text a computer shows when something fails. It tells you what went wrong

Bad and good prompts for the 3 tasks

1. Bash script

```
BAD:
Write a backup script.

GOOD:
Role: You are a friendly Linux teacher.
Context: I am a beginner. My computer is [your system, e.g. Ubuntu, Windows, Mac].
Task: Write a small script that makes a backup copy of my folder called "projects".
Constraints: Do not delete anything. Keep it short.
Format: Show the script, then explain each line in simple words.
```

The bad prompt makes Claude guess: which OS, which folder, where to copy, whether to overwrite or delete, how to test.

2. YAML file (Kubernetes Deployment)

```
BAD:
Write YAML for nginx.

GOOD:
Role: You are a friendly Kubernetes teacher.
Context: I am a beginner. I only want to read the file, not run it.
Task: Write a YAML file that runs 2 copies of the nginx web server.
Constraints: Use a fixed version number, not "latest". Add nothing extra.
Format: Show the YAML, then explain each part in simple words.
```

The bad prompt leaves out how many copies, which version (it may use "latest"), and may add extra things. The good prompt asks for 2 copies, a fixed version and nothing extra.

3. Explain an error log

```
BAD:
What is wrong here? Failed to pull image nginx:1.99.9

GOOD (with XML tags):
<task>Explain this error in simple words. Tell me the likely cause and how to fix
it.</task>
<error>
Failed to pull image "nginx:1.99.9": manifest unknown
Error: ImagePullBackOff
</error>
<constraints>Do not guess. Say if you are not sure.</constraints>
```

The error means Kubernetes could not download the app, because the version 1.99.9 does not exist. A good answer says that, and says to use a version that exists. Check Claude: does the answer match the error, and did it say anything that is not in the error?

XML tags: what they are and why we use them

A tag is a label written in angle brackets. **<task>** opens a box and **</task>** closes it (note the slash). Everything between them belongs to that label. You choose the names: task, error, log, rules.

```
<task>Explain this error in plain English.</task>
<error>
Failed to pull image
</error>
```

Purpose	What it does for you
1. Separate	Your request in one box and the pasted text in another. Claude does not mix them up.
2. Label	When you paste several things, name each one, for example <old> and <new>. Then you can say "compare old and new".
3. Protect	Pasted text can contain sentences that look like orders, such as "ignore the above". Inside a box, Claude treats it as data, not as your instruction.
4. Reuse	Keep one template. Next time only swap what is inside the boxes.
5. Find the answer	Ask: "Put your answer inside <answer> tags." The answer is easy to find and copy.

```
Without tags:
Explain this error in simple words and tell me the likely cause and how to fix it.
Failed to pull image "nginx:1.99.9": manifest unknown Error: ImagePullBackOff and
do not guess

With tags:
<task>Explain this error in simple words. Tell me the likely cause and how to fix
it.</task>
<error>
Failed to pull image "nginx:1.99.9": manifest unknown
Error: ImagePullBackOff
</error>
<constraints>Do not guess. Say if you are not sure.</constraints>
```

Be honest: tags are not magic and not required. A plain label like "ERROR:" also works. Tags help most when you paste long text or more than one thing.

Few-shot: show examples, get the same shape every time

Few-shot means you give a few examples of the answer you want. Without examples Claude may write a long paragraph, and the shape changes each time. With 2 examples, Claude copies the pattern and answers in the same 3 lines: Error, Meaning, Fix.

```
Explain each error in plain English, and give a fix. Follow the two examples.

Example 1
Error: No space left on device
Meaning: The disk is full.
Fix: Remove files you do not need.

Example 2
Error: Permission denied
Meaning: You are not allowed to do this.
Fix: Check your user or ask for access.

Now do this one:
Error: Connection refused
```

Step by step: ask for the reasons in order

Use this when a problem has many possible causes. Without steps, Claude gives one guess and you cannot check it. With steps, Claude gives 3 likely reasons one by one and tells you how to check each one.

```
My CI pipeline passed yesterday and fails today. I only changed one setting. Give me the 3 most likely reasons, one by one, and one way to check each. Show your steps.
```

Explain vs execution-ready

```
Explain this command line by line for a beginner. Do not change it.
ls -l
```

Explain prompts are read-only and safe. **Execution-ready** prompts ask for code with comments, error checks and how to test and undo it. Claude only writes the code as text; it does not run it. Read every line and run it yourself on a practice machine. After any code, ask:

```
What could go wrong if I run this? What should I check first, and how can I undo it?
```

Peer-review checklist

Check	Question
Role	Does it say who Claude should be?
Context	Did you tell Claude what it cannot see?
Task	Is it clear what you want?
Constraints	Does it say what NOT to do?
Format	Is the shape of the answer clear?
Verify	Did you ask what could go wrong?

Key words

Word	Meaning
Prompt	The instructions and data you send to Claude
Context	Facts about your situation that Claude cannot see
Constraint	A rule Claude must follow, such as "do not delete anything"
Few-shot	Giving a few examples so Claude copies the pattern
XML tags	Labels like <log> that separate your instructions from pasted data
YAML	A settings file that describes what Kubernetes should run

Check yourself

1. Why does Claude need you to type your operating system and folder names? Can Claude on the free web chat create a folder for you?
2. What do the letters R-C-T-C-F stand for?
3. Why is "free up disk space on my computer" a risky prompt?
4. What does few-shot mean?
5. What question should you ask after Claude gives you code?

Answers

1. Claude cannot see your computer. It only knows what you type, otherwise it guesses. It also cannot create folders or run anything: it only writes text and you run the code yourself.
2. Role, Context, Task, Constraints, Format.
3. Claude may decide what to delete. Ask for a list first and say do not delete anything.
4. Giving a few examples so Claude follows the pattern.
5. What could go wrong if I run this, and what should I check first?

Take-home task (20 to 30 minutes)

1. Take the 3 infra tasks you listed on Day 2.
2. For each, write a bad prompt and a good prompt using the template.
3. Run both, compare the answers, and ask what could go wrong.
4. Bring your best prompt to Day 4. No passwords, secrets or company data in any chat.

> **Tonight, before Day 4:** subscribe to Claude Pro so it is active before class (check the current price and plan details on claude.com/pricing first). Install Claude Desktop (Windows or Mac) and sign in with the same account. If you cannot subscribe, message the instructor before Day 4.